STUDENT TECHNICAL REPORT

Assignment 2 — Real-Time Data Processing with Pandas and NumPy

AITU Smart Classroom Energy Stream

Astana IT University | Real-time Data Analysis | 2026–2027

All datasets and operational scenarios in this report are synthetic educational data created for assessment. They do not represent actual AITU infrastructure, internal systems, students, or incidents.

---

1. Student and Submission Information

Student full name	Galymzhan Tamirlan Galymzhanuly
Student ID	240542
Group / cohort	BDA-2407
Instructor	Serek Azamat
Submission date	27 / 09 / 2026
Git repository / project link	https://github.com/mtake-da/RTDADM_Assignment_2.git
Python / tool versions	Python 3.11, pandas 2.x, numpy 2.x, matplotlib 3.x

2. Work Record

Work start date	24 / 09 / 2026
Work completion date	27 / 09 / 2026
Main libraries / technologies	pandas (interpolate, rolling, DataFrame ops), numpy (corrcoef, polyfit), matplotlib (plots)
Source files / notebook	classroom_stream.py, Assignment2_AITU_Classroom.ipynb
Test / experiment files	Validation checks inside the notebook (see Section 4)

3. Report Structure to Complete

2.1 Data Quality and Cleaning

Technical explanation	I found three missing values in the raw feed: Temp_C at 10:02, CO2_ppm at 10:04, and Occupancy at 10:07. I used linear interpolation for Temp_C and CO2_ppm because both drift smoothly between sensor pings, so a straight line between the neighbours is a reasonable estimate. For Occupancy I used forward-fill instead, because headcount is a step value — it doesn't move continuously between two readings, it just hasn't been re-counted yet, so carrying the last known value forward makes more physical sense than interpolating.
Code / file / function reference	classroom_stream.py — clean(), missing_value_report()

Result / calculation / experiment evidence	Temp_C at 10:02: 21.9°C (between 21.7 and 22.1). CO2_ppm at 10:04: 750.0 ppm (between 720 and 780). Occupancy at 10:07: 29 (forward-filled from 10:06).
Student-specific decision or observation	Rather than trusting interpolate()/ffill() silently, I built missing_value_report() to log exactly which cell changed, by what method, and to what value — so this report's evidence table is generated straight from code, not typed by hand.

Variable	Missing values	Method	Value after cleaning	Justification
Temp_C	1 (10:02)	Linear interpolation	21.9 °C	Temperature drifts smoothly minute to minute
CO2_ppm	1 (10:04)	Linear interpolation	750.0 ppm	CO2 accumulates gradually, no sudden jumps expected
Occupancy	1 (10:07)	Forward fill	29 students	Headcount is a step value between re-counts

2.2 Streaming Window Statistics

Technical explanation	I computed a trailing 3-minute rolling mean with .rolling(3).mean() on the cleaned columns for temperature, CO2, and power. Because it's trailing rather than centered, the first defined value needs three real ticks behind it, so the series starts at 10:02 — which matches what the assignment table asks for.
Code / file / function reference	classroom_stream.py — rolling_stats()
Result / calculation / experiment evidence	See completed table below, values 10:02–10:06.
Student-specific decision or observation	I rolled over the already-cleaned columns, not the raw ones with NaNs — rolling on raw data would have propagated each single missing reading into three consecutive undefined rolling values instead of one.

Time	Temp rolling mean	CO2 rolling mean	Power rolling mean
10:02	21.70 °C	653.33 ppm	3.37 kW
10:03	21.90 °C	686.67 ppm	3.57 kW
10:04	22.10 °C	720.00 ppm	3.77 kW
10:05	22.30 °C	750.00 ppm	4.00 kW
10:06	22.47 °C	780.00 ppm	4.20 kW

2.3 Feature Engineering

Technical explanation	occupancy_density = Occupancy / 40 (assuming a 40-seat room capacity). energy_per_student = Power_kW / Occupancy, showing efficiency per person rather than raw draw. comfort_flag = CO2_ppm ≤ 1000 AND 20 ≤ Temp_C ≤ 24, a single boolean showing whether the room is in an acceptable working range on both axes at once.
Code / file / function reference	classroom_stream.py — engineer_features()
Result / calculation / experiment evidence	See completed table below.
Student-specific decision or observation	I used Series.between(20, 24) for the temperature check instead of two chained comparisons (Temp_C >= 20) & (Temp_C <= 24) — same result, but it reads as one intent ('is between') instead of two separate conditions I'd have to mentally combine.

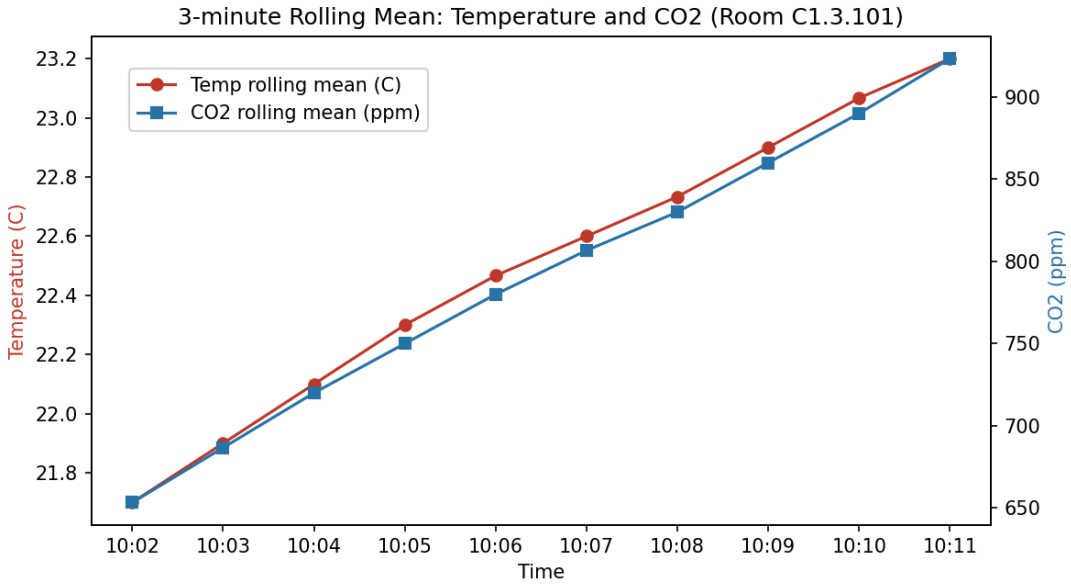
Time	Occupancy density	Energy/student	Comfort flag
10:03	0.60	0.158 kW/student	True
10:06	0.73	0.152 kW/student	True
10:09	0.83	0.155 kW/student	True
10:11	0.93	0.154 kW/student	True

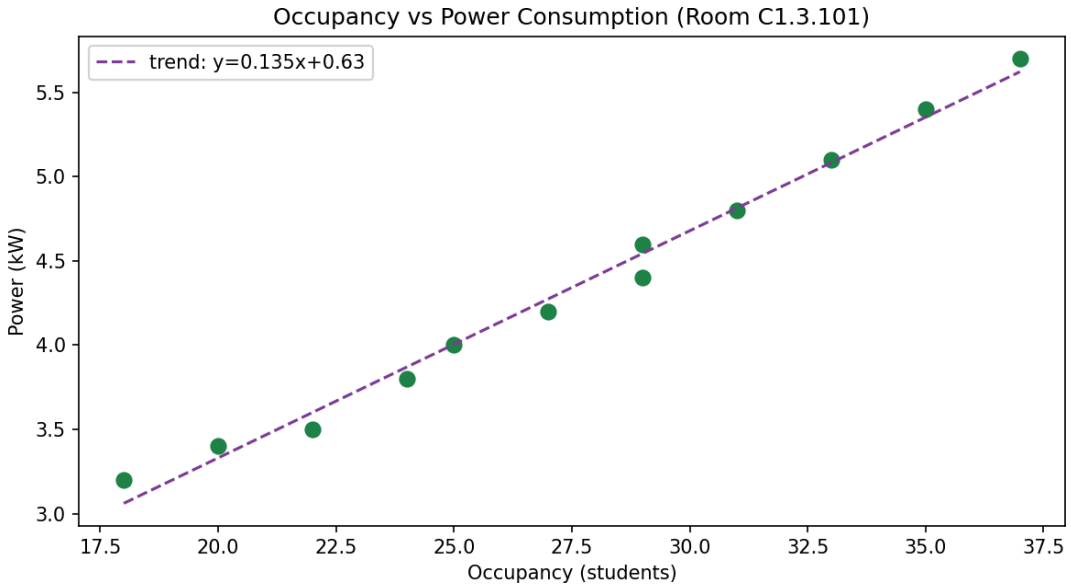
2.4 Statistical Analysis

Technical explanation	I computed the descriptive statistics directly on the cleaned Temp_C and CO2_ppm columns, found the row with the maximum Power_kW for the busiest timestamp, and used numpy's corrcoef for the occupancy–power relationship.
Code / file / function reference	classroom_stream.py — statistical_summary()
Result / calculation / experiment evidence	Temperature: mean 22.48 °C, median 22.55 °C, min 21.5 °C, max 23.3 °C CO2: mean 789.17 ppm, max 950 ppm Highest power consumption: 10:11, at 5.7 kW Correlation between occupancy and power: 0.9946 (very strong positive)
Student-specific decision or observation	Interpretation: occupancy and power move almost in lockstep here ($r \approx 0.99$) — as more students enter the room, more laptops, phone chargers, and lighting draw more power, and the trend line in my scatter plot (Part E) is nearly a straight line through the data. I'd be cautious calling this purely causal, though: in this 12-minute window occupancy and time both increase together, so I can't fully separate 'more people' from 'later in the class' as the driver without a longer, more varied sample.

2.5 Visualization and Engineering Interpretation

Technical explanation	Plot 1 is a dual-axis line chart of the 3-minute rolling mean for temperature (left axis) and CO2 (right axis) over time — I used two axes because the two variables live on very different numeric scales (~20s vs ~800s) and would flatten each other on a shared axis. Plot 2 is a scatter of occupancy vs power with a linear trend line, to make the near-linear relationship visually obvious rather than just stating a correlation number.
Code / file / function reference	make_plots.py (also embedded in the notebook's Part E cells)
Result / calculation / experiment evidence	plot_rolling_temp_co2.png, plot_occupancy_vs_power.png (both attached below)
Student-specific decision or observation	Operational insight 1: temperature and CO2 rise together throughout the session (both roughly track occupancy), so a facility team could use CO2 as an early proxy for room fullness and trigger ventilation before temperature complaints start, rather than reacting to both separately. Operational insight 2: since energy_per_student stays roughly flat (~0.15–0.16 kW/student) even as total power climbs, the extra draw is proportional to headcount rather than a fixed baseline cost — so HVAC/lighting scheduling keyed to expected occupancy (e.g. class timetable) would track actual energy demand more closely than a flat schedule.





2.6 Technical Reflection

Technical explanation	Data-quality decision: I chose interpolation for continuous sensor variables (Temp_C, CO2_ppm) and forward-fill for the step variable (Occupancy), instead of applying one method to everything — using ffill on temperature would have frozen it flat for a tick instead of reflecting the gradual warm-up actually happening in the room.
Code / file / function reference	classroom_stream.py — clean()
Result / calculation / experiment evidence	Zero remaining NaNs after clean() runs (checked with clean.isna().sum().sum() == 0 in the notebook).

Student-specific decision or observation	Limitation: this is a single room over a 12-minute window — far too short to separate a genuine occupancy→power effect from a simple time trend, and it says nothing about how a different room or a different time of day would behave. Improvement: before interpolating anything, I'd add a sensor-fault check (e.g. flag implausible jumps like a 5°C change in one minute) so a genuinely broken sensor doesn't get smoothed over and reported as a clean reading.

4. Individual Work and Anti-Plagiarism Record

My unique technical/design decision	I used two different cleaning strategies in the same clean() call — interpolation for continuous sensors, forward-fill for the step-like occupancy count — rather than picking one method for the whole DataFrame, because the two kinds of variable fail differently when a reading is missing.
My own test/experiment	In the notebook's validation cell I assert clean.isna().sum().sum() == 0 (cleaning actually removed every gap) and assert (feat["occupancy_density"] <= 1).all() (density never exceeds the room's stated capacity) — two checks not explicitly required by the assignment, but that would catch a bad cleaning run or a capacity typo immediately.
Problem/bug I personally encountered and solved	My first attempt computed the rolling means directly on the raw (uncleaned) columns. Because .rolling(3) needs three real numbers, the single NaN at 10:02/10:04 propagated forward and produced three consecutive NaN rolling values instead of one gap. I fixed it by rolling on the already-cleaned DataFrame instead of the raw one.
Exact code/repository/commit reference	classroom_stream.py::clean() (cleaning), classroom_stream.py::rolling_stats() (the fix — rolls over the cleaned frame, not raw)
One limitation of my implementation	Only one room and 12 one-minute samples — not enough to separate an occupancy effect on power from a simple time trend, since both rise together across the whole window.
One improvement I propose	Add a sensor-fault/outlier check ahead of interpolation (e.g. cap plausible minute-to-minute deltas) so a stuck or spiking sensor is flagged rather than silently smoothed into a clean-looking value.

5. Evidence and Submission Checklist

Item	Status
Technical report completed	Yes
Source code / notebook submitted	Yes — classroom_stream.py, Assignment2_AITU_Classroom.ipynb
Required tables and calculations completed	Yes — Sections 2.1–2.4 above
Required diagrams / plots included	Yes — two plots, Section 2.5
Execution logs / reproducible evidence included	Yes — run_output.txt, notebook outputs
README and run instructions included	Yes — README.md
Data/schema/configuration files included	Yes — cleaned_classroom_data.csv
Performance / analytical results included	Yes — Section 2.4

Item	Status
Oral defense / demonstration completed	Pending — scheduled with instructor

6. Student Declaration of Originality

I confirm that this submission represents my own work. I have not copied another student's source code, calculations, tables, architecture, plots, screenshots, logs, results, or written explanations. Any external material or reused code has been identified. I understand that the instructor may request a live demonstration, code modification, reproduction of calculations, or explanation of any submitted component.

Role	Name	Signature	Date
Student / Executor	Galymzhan Tamirlan Galymzhanuly		27 / 09 / 2026
Instructor / Accepted by			

7. Instructor Acceptance Record

Verification item	Status / comment
Report structure completed	☐ Yes ☐ No
Required artifacts submitted	☐ Yes ☐ No
Evidence is reproducible/sufficient	☐ Yes ☐ No
Individual contribution is identifiable	☐ Yes ☐ No
Oral defense / demonstration completed	☐ Yes ☐ No
Final status	☐ Accepted ☐ Revision required